Plano de testes

GameStore - Recuperação / Etapa 7
Lucas Oliveira | 04 de novembro de 2026 | Versão 1.0

1. Objetivo e escopo

Planejar e verificar as principais regras da GameStore criada na Etapa 6. Os testes unitários JUnit concentram-se no cálculo de vendas, validação de produtos e casos de uso do serviço. O plano manual contempla a aplicação atual e os fluxos da futura interface web.

A aplicação atual é Java SE com console e persistência local. A interface web ainda não foi implementada. Cenários web são planejamento e não são apresentados como testes executados.

2. Ambiente e preparação

Item	Configuração
Aplicação	Fontes reais de GameStore/src; regras sem dependência de Swing.
Projeto de testes	GameStoreTestes, Java SE/Ant, JDK 17+, JUnit 4.13.2 e Hamcrest Core 1.3 incluídos.
Ambiente executado	Linux, OpenJDK 17.0.20 e Apache Ant 1.10.15; UTF-8.
Isolamento	MemoryRepository por cenário e Clock fixo em 04/11/2026 12:00 UTC. Nenhum teste JUnit acessa dados/gamestore.dat ou banco de dados.
Reprodução	Manter as pastas GameStore e GameStoreTestes lado a lado. Executar ant clean test em GameStoreTestes. No NetBeans: abrir GameStoreTestes e usar Testar Projeto ou Executar.

3. Estratégia e resultado

Particionamento de equivalência: dados válidos e inválidos; valores-limite: zero, estoque exato, estoque excedido, centavos e quantidade inteira máxima. O serviço é testado também com falha de gravação simulada para garantir que a venda não seja confirmada parcialmente.

Execução automatizada concluída: 28 testes, 0 falhas e 0 ignorados. Compilação com -Xlint:all e -Werror, sem avisos. Saída completa: evidencias/execucao-junit.txt; relatório por método: evidencias/TEST-GameStoreJUnit.xml.

Não foi necessário alterar o código de produção: Sale.total() já isola o cálculo monetário. A interface gráfica do NetBeans não foi aberta neste ambiente; o build Ant foi compilado e executado diretamente.

4. Cobertura automatizada e rastreabilidade

Todos os grupos abaixo foram executados e aprovados. Os nomes exatos dos 28 métodos constam no relatório XML e nos fontes anotados com @Test.

IDs / classe	Dados ou operação	Resultado esperado / quantidade
U01-U04 / SaleTest	2 × 99,90; 3 × 0,10; 1 × 59,99; 2.147.483.647 × 99.999.999,99.	199,80; 0,30; 59,99; 214.748.364.678.525.163,53, sem perda decimal ou overflow inteiro. 4 testes.
U05-U07 / SaleTest	Quantidade zero/negativa e preço zero/negativo.	BusinessException para dados inválidos. 3 testes.
U08-U09 / ProductTest	Nome com espaços externos; estoque zero e preço 0,01.	Nome normalizado e estoque zero aceito. 2 testes.
U10-U14 / ProductTest	Nome em branco, estoque -1, preço zero, negativo ou com 3 casas.	BusinessException para cada categoria inválida. 5 testes.
U15 / ProductTest	Preço inteiro 10 e estoque válido.	Preço aceito sem conversão para double. 1 teste.
U16-U17 / StoreServiceTest	Venda de 2 unidades; venda de todo o estoque.	Total correto, registro com data fixa, estoque reduzido; venda seguinte sem saldo rejeitada. 2 testes.
U18-U20 / StoreServiceTest	Quantidade maior que estoque; zero/negativa; produto inexistente.	Rejeição sem gravar venda nem alterar estoque. 3 testes.
U21 / StoreServiceTest	Novo produto com mesmo nome, caixa distinta e espaços.	Cadastro duplicado rejeitado. 1 teste.
U22-U24 / StoreServiceTest	Alteração válida; alteração para nome duplicado; preço zero na alteração.	Identificador preservado; alterações inválidas mantêm produto anterior. 3 testes.
U25-U26 / StoreServiceTest	Alterar/excluir produto vendido; tentar modificar listas retornadas.	Histórico preserva nome/preço e listas rejeitam alterações externas. 2 testes.
U27-U28 / StoreServiceTest	save lança IOException; reabrir serviço com mesmo repositório em memória.	Falha mantém estoque/histórico; nova instância recupera estado salvo em memória. 2 testes.

5. Critérios de aceite automatizados

A suíte deve compilar, executar os 28 testes sem falhas ou ignorados e produzir o relatório XML. Uma falha de asserção causa código de saída diferente de zero. A suíte deve continuar independente de banco, rede e dados reais da loja.

6. Testes manuais da aplicação atual (1/2)

Executar em uma pasta de dados de teste, separada do cadastro real. Usar Java 17+ e o JAR da GameStore. Os procedimentos abaixo são planejados para execução pelo responsável pela validação.

M01 - Cadastro e consulta

Prioridade: Alta. Situação: planejado, não executado manualmente nesta etapa.

Pré-condições: Aplicação iniciada sem produtos na pasta de teste.

Passos: 1) Selecionar 2. 2) Informar Jogo A, preço 99,90 e estoque 5. 3) Selecionar 1. 4) Anotar o ID exibido.

Resultado esperado: Mensagem de cadastro; listagem com nome Jogo A, preço 99,90, estoque 5 e ID não vazio.

M02 - Rejeição de dados inválidos

Prioridade: Alta. Situação: planejado, não executado manualmente nesta etapa.

Pré-condições: Cadastro de M01 concluído.

Passos: Repetir cadastro com nome vazio; depois preço 0; preço -1; preço 1,001; estoque -1. Listar após cada tentativa.

Resultado esperado: Cada entrada inválida é rejeitada com mensagem; não surge produto adicional e o cadastro anterior permanece.

M03 - Nome duplicado

Prioridade: Alta. Situação: planejado, não executado manualmente nesta etapa.

Pré-condições: Jogo A cadastrado.

Passos: Selecionar 2 e informar nome jogo a com espaços externos, preço 10 e estoque 1. Em seguida, listar produtos.

Resultado esperado: Duplicidade rejeitada ignorando caixa e espaços; existe somente o produto original.

M04 - Alteração do cadastro

Prioridade: Alta. Situação: planejado, não executado manualmente nesta etapa.

Pré-condições: ID do produto de M01 conhecido.

Passos: Selecionar 3; informar o ID, nome Jogo B, preço 120,00 e estoque 4. Selecionar 1.

Resultado esperado: Produto exibido com os novos dados e o mesmo ID; nenhuma segunda linha é criada.

6. Testes manuais da aplicação atual (2/2)

M05 - Venda e cálculo

Prioridade: Alta. Situação: planejado, não executado manualmente nesta etapa.

Pré-condições: Produto com preço 99,90 e estoque 5 em massa de teste reinicializada.

Passos: Selecionar 5; informar ID e quantidade 2. Consultar produtos (1) e vendas (6).

Resultado esperado: Total 199,80; estoque 3; histórico com 2 unidades, nome e preço originais e data da venda.

M06 - Limite do estoque e quantidade inválida

Prioridade: Alta. Situação: planejado, não executado manualmente nesta etapa.

Pré-condições: Produto com estoque 3.

Passos: Tentar vender 4, depois 0 e -1. Consultar estoque/histórico. Depois vender 3 e tentar vender mais 1.

Resultado esperado: Tentativas inválidas não alteram dados; venda de 3 zera estoque; nova venda é rejeitada.

M07 - Preservação do histórico

Prioridade: Alta. Situação: planejado, não executado manualmente nesta etapa.

Pré-condições: Há uma venda registrada para um produto conhecido.

Passos: Alterar nome e preço do produto. Consultar vendas; excluir o produto pela opção 4 e consultar novamente as vendas.

Resultado esperado: Produto sai da listagem; histórico permanece com nome, preço, quantidade e total da época da venda.

M08 - Persistência após reiniciar

Prioridade: Alta. Situação: planejado, não executado manualmente nesta etapa.

Pré-condições: Produto e venda registrados em pasta de teste.

Passos: Sair pela opção 0. Abrir novamente a aplicação na mesma pasta de trabalho. Consultar produtos e vendas.

Resultado esperado: Estado recuperado: produtos, estoque atualizado e histórico equivalentes aos dados anteriores. Nenhum cadastro duplicado.

7. Testes planejados para a interface web

Todos os casos W01-W05 estão planejados e dependem da implementação futura da interface HTTP. Não existe resultado de execução web nesta entrega. Ambiente previsto: servidor local, navegador atualizado e base de testes isolada.

W01 - Cadastro e atualização da listagem

Prioridade: Alta. Situação: planejado, não executado manualmente nesta etapa.

Pré-condições: Página web de produtos disponível e base de testes vazia.

Passos: Abrir cadastro; preencher Jogo A, 99,90 e estoque 5; salvar; voltar à listagem e recarregar a página.

Resultado esperado: Produto aparece uma única vez; dados são recuperados após recarregar; mensagem de sucesso visível.

W02 - Validação no cliente e no servidor

Prioridade: Alta. Situação: planejado, não executado manualmente nesta etapa.

Pré-condições: Formulário web e endpoint de cadastro implementados.

Passos: Enviar nome vazio, preço zero e estoque negativo pelo formulário e por requisição direta ao endpoint.

Resultado esperado: Interface informa os erros; servidor também rejeita cada entrada e não persiste produto inválido.

W03 - Venda com saldo insuficiente

Prioridade: Alta. Situação: planejado, não executado manualmente nesta etapa.

Pré-condições: Produto com estoque 1 disponível na interface web.

Passos: Tentar vender 2; consultar estoque e histórico; em seguida vender 1.

Resultado esperado: Primeira tentativa exibe erro e mantém dados; segunda registra a venda e atualiza estoque para zero.

W04 - Edição, exclusão e histórico

Prioridade: Alta. Situação: planejado, não executado manualmente nesta etapa.

Pré-condições: Produto vendido e telas web de produto e histórico implementadas.

Passos: Alterar nome/preço; conferir histórico; excluir produto; recarregar histórico.

Resultado esperado: Produto deixa a listagem; vendas mantêm os dados originais, sem depender de produto ainda ativo.

W05 - Usabilidade da listagem e mensagens

Prioridade: Média. Situação: planejado, não executado manualmente nesta etapa.

Pré-condições: Páginas web implementadas.

Passos: Consultar lista vazia; cadastrar produto; abrir em janela de 360 px e em desktop; navegar por teclado.

Resultado esperado: Estado vazio compreensível; campos e botões legíveis; sem rolagem horizontal indevida; foco visível e operação por teclado.

8. Execução, registro e encerramento

Responsável previsto: Lucas Oliveira. Preparar a massa, executar os passos na ordem descrita e comparar o observado ao esperado. Registrar cada caso como aprovado, reprovado ou bloqueado, com data, ambiente e evidência. Em cenários web, registrar também o navegador e a versão da aplicação.

Campo de execução	Informação a registrar
Identificação	ID do caso, versão/commit, data e responsável.
Condições	Ambiente, pré-condições e dados efetivamente utilizados.
Resultado	Resultado observado, resultado esperado e situação final.
Evidência	Captura da tela, saída de execução ou identificação de defeito.
Defeito	Passos para reprodução, severidade, comportamento observado e esperado.

9. Critérios de entrada e saída

Entrada: versão compilável, dependências disponíveis, massa de teste isolada e funcionalidades acessíveis no ambiente correspondente. Um caso web só entra em execução após a respectiva tela e endpoint existirem.

Saída da validação atual: 28 testes JUnit aprovados e nenhum defeito crítico de cálculo, estoque ou integridade. Para homologação manual futura, todos os casos de prioridade alta aplicáveis devem passar; qualquer bloqueio deve ser registrado, sem ser tratado como aprovação.

Severidade crítica: perda de dados, total monetário incorreto ou estoque inconsistente. Alta: fluxo principal indisponível ou validação burlável. Média: apresentação ou navegação com alternativa de uso. Correções exigem repetição do caso afetado e execução da suíte JUnit para regressão.

10. Limites e riscos

A suíte unitária usa armazenamento em memória e não certifica concorrência, disco, banco de dados, rede, autenticação ou desempenho. Persistência em arquivo é prevista em M08; a aplicação atual deve ter apenas uma instância por arquivo. Testes web não autorizam afirmar que um sistema web já foi implementado.

11. Evidências e versionamento

Os resultados automatizados estão em evidencias/execucao-junit.txt e evidencias/TEST-GameStoreJUnit.xml. O projeto de testes foi publicado no mesmo repositório da Etapa 6. O documento Evidencia_GitHub_Etapa7.pdf contém links, SHA do commit e capturas reais da publicação.